

AI Coding with Junie

JetBrains' Coding Agent for IDE, CLI, and CI

Plan, implement, review, and verify
with a coding agent that understands your project

What You'll Master Today

Core Concepts

-  **Ask Mode:** Read-only exploration
-  **Plan Mode:** Align before coding
-  **Code Mode:** Implement and verify
-  **Safety:** Approvals, allowlists, review

Hands-On Labs

-  **Java:** Spring Boot APIs
-  **Python:** Refactoring & testing
-  **React:** Forms & validation
-  **MCP:** Current docs and tools



Prerequisites Check

- ✓ **JetBrains IDE** (IntelliJ IDEA, PyCharm, WebStorm, GoLand, PhpStorm, RustRover, RubyMine)
- ✓ **JetBrains AI** subscription, Junie API key, or BYOK provider key
- ✓ **Development Environment** (Java 21+ / Python 3.8+ / Node 18+)

Cost control: Use `/usage``, choose models intentionally, and prefer BYOK/local models for heavy practice



Quick Setup: We'll verify Junie, CLI, and lab runtimes together at the start

Getting Started — Two Surfaces

JetBrains IDE

1. AI Chat → Agent dropdown → ****Junie****
2. Separate Junie tool window remains available
3. Best for indexes, refactorings, debugger, inspections

Junie CLI

1. ``brew tap jetbrains-junie/junie && brew install junie``
2. Or: ``curl -fsSL https://junie.jetbrains.com/install.sh | bash``
3. Run ``junie`` in any project, or ``junie "task"`` headlessly

Model and provider options:

JetBrains account / credits

OpenAI / Anthropic / Google / xAI

OpenRouter / Copilot

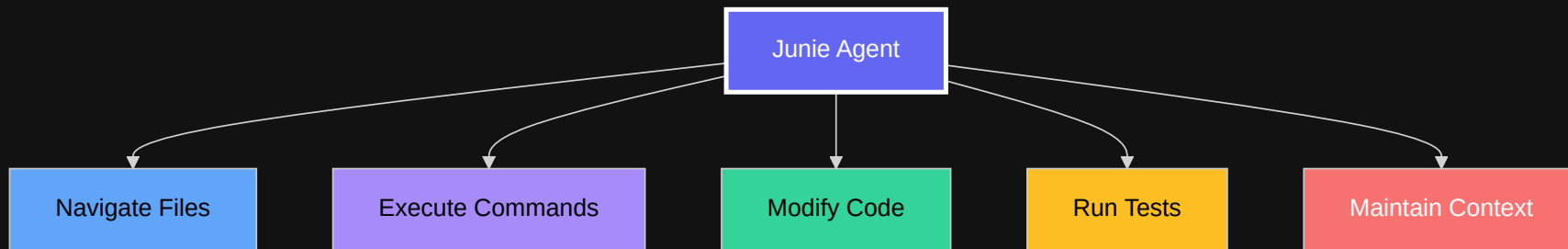
Ollama / LM Studio / LiteLLM

Use ``/model`` and ``/usage`` to stay intentional



What is Junie?

A coding agent that works where JetBrains developers already work



NEW

Junie After Beta

General Availability Story

- JetBrains now calls Junie a coding agent
- IDE, terminal, and CI/CD are one product story
- Plans before it codes
- Runs long tasks while you review intent

Plan Mode Is Central

- Read-only exploration first
- Requirements, design, tests, delivery steps
- Review and refine before implementation
- Save plans as durable task docs

NEW

Junie After Beta (continued)

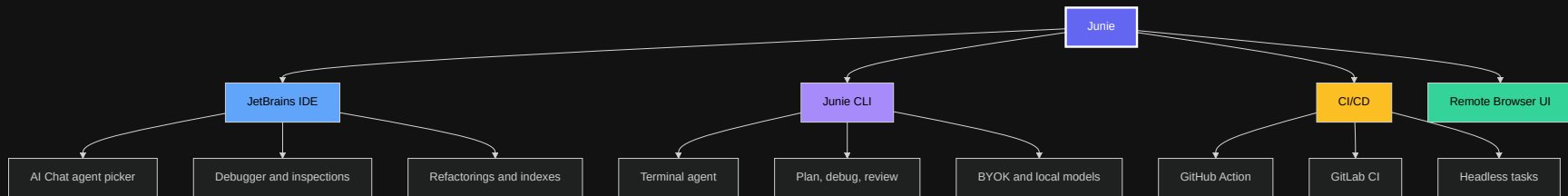
Shared Agent Context

- ``.junie/AGENTS.md`` first
- Root ``AGENTS.md`` also supported
- Skills live in ``.junie/skills/``
- Extensions bundle skills, MCP, commands, agents

Beyond the IDE

- ``/review`` for local diffs
- ``/debug`` with JetBrains debugger context
- ``/remote`` to continue a CLI session in browser
- GitHub Action and GitLab CI workflows

Junie Surfaces



The CLI can connect to a matching JetBrains IDE for symbol-aware search, inspections, test discovery, and debugger-backed workflows.



Three Working Modes



Ask Mode

Read-Only Analysis

- Explain complex code
- Analyze architecture
- Find bugs & issues
- Review security
- Check test coverage

Perfect for understanding before changing



Plan Mode

Design Before Edits

- Requirements
- Technical design
- Testing strategy
- Delivery steps
- Confirm or refine

Best for non-trivial tasks

⚡ Code Mode

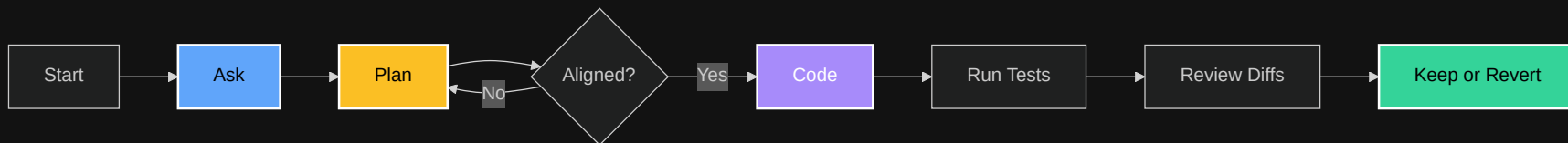
Make Changes

- Implement features
- Fix bugs
- Refactor code
- Generate tests
- Update dependencies

Executes multi-step plans with diffs



Professional Workflow



Use Plan mode when the cost of the wrong implementation is higher than the cost of five minutes of alignment.

Plan Mode in Practice

Start It

- Press `Shift+Tab` in Junie CLI
- Or type `/plan <task>`
- In the IDE, start with Ask/Auto and request a plan first
- For installed flags, check `junie --help`

Review It

- Read assumptions
- Edit scope before code exists
- Check testing strategy
- Confirm implementation only after the plan matches your intent

Teaching move: use one complete prompt, let Junie plan, then ask attendees what they would change before implementation.



Project Guidelines

Preferred: `.junie/AGENTS.md` | **Also supported:** `root AGENTS.md` | **Legacy:**
`.junie/guidelines.md`

```
## Technology Stack
- Framework: Spring Boot 3.2
- Testing: JUnit 5 + AssertJ

## Conventions
- REST: /api/v1/{resource}
- DTOs: Java records
- Tests: Given-When-Then
```

Lookup order: ``.junie/AGENTS.md`` → ``AGENTS.md`` (project root) → ``.junie/guidelines.md`` (legacy). Global guidelines can live in `~/junie/AGENTS.md``.

✗ Without Guidelines

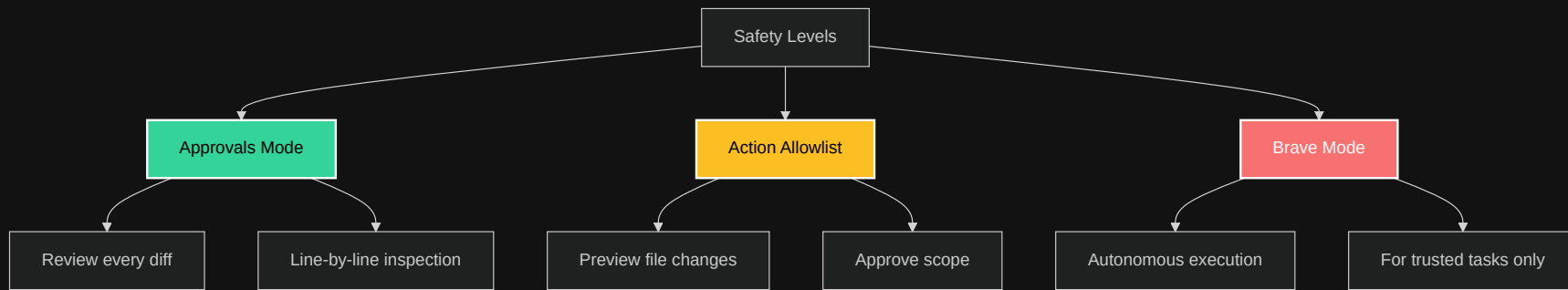
Inconsistent patterns
Multiple review cycles
Style conflicts

✓ With Guidelines

Consistent output
Fewer review cycles
Team alignment



Safety Controls



⚡ When to Use Each Mode

Mode	Good For	Not For
Approvals	• Critical business logic • First-time tasks • Learning Junie	• Repetitive formatting • Simple refactors
Allowlist	• Known file sets • Defined scope • Team reviews	• Exploratory changes • Unknown impact
Brave	• Test generation • Formatting • Documentation	• Production code • Database changes • First attempts

MCP: Model Context Protocol

Use MCP For

- External services
- Current docs
- Browser exploration
- Shared project tools

Use Skills For

- Repeatable workflows
- Test conventions
- Review checklists
- Team patterns

```
{
  "mcpServers": {
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp"]
    },
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp"]
    }
  }
}
```

Where it lives

CLI: ``/mcp``

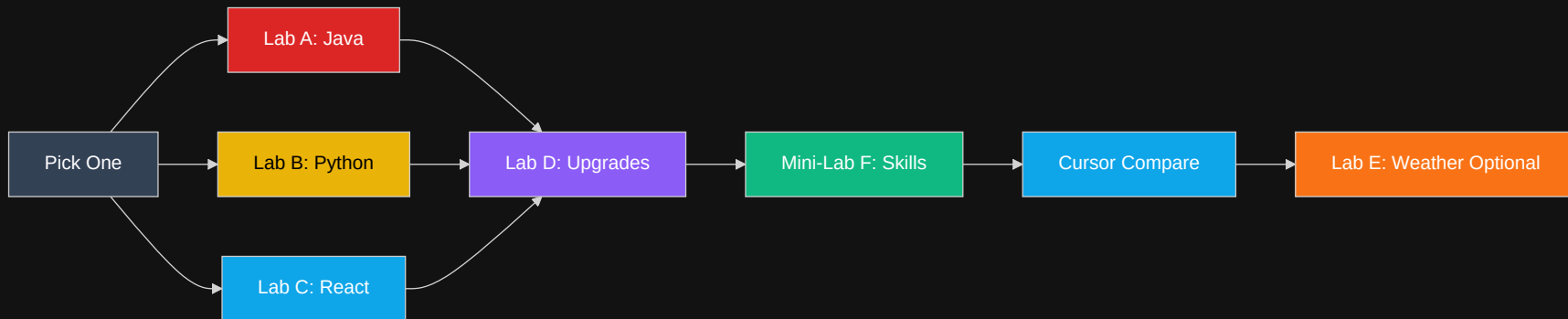
Project: ``.junie/mcp/mcp.json``

User: ``${~}/.junie/mcp/mcp.json``

context7 is the main workshop MCP example.



Lab Overview



Java

Spring Boot

REST APIs

JUnit + AssertJ



Python

PEP 8

Type hints

pytest



React

TypeScript

Forms

Testing Library



MCP

context7



Skills

Reusable



Lab E

Weather API

Lab A: Spring Boot Journey



Baseline Pass

- Ask Junie to inspect first
- Let it propose a plan
- Watch where style drifts

Guided Pass

- ``.junie/AGENTS.md`` loaded
- One complete feature prompt
- Tests and review included

Lab B: Python Transformation

Starting Point

```
def calc(x,y,op):  
    if op=="add": return x+y  
    elif op=="sub": return x-y
```

- No type hints
- Poor naming
- No tests
- No docs

Target Outcome

```
def calculate(  
    left: float,  
    right: float,  
    operation: Operation,  
) -> float:  
    """Apply a supported operation."""
```

- Type-safe API
- pytest coverage
- Documented edge cases



Lab C: React Forms

```
interface RegistrationForm {  
  email: string;    // valid email  
  password: string; // min 8, special char  
  confirm: string;  // must match  
  terms: boolean;   // required  
}
```

React Hook Form
Form state

Zod
Validation

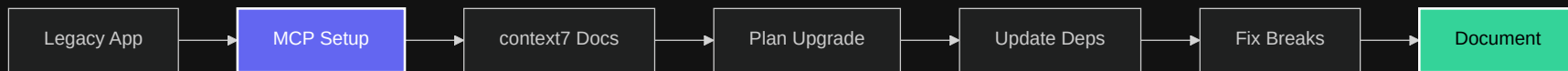
Testing Library
90% coverage

Accessibility
ARIA compliant

Teaching move: Use one complete prompt that asks for the component, validation, accessible errors, tests, and a final self-review.



Lab D: Smart Upgrades with context7



React 17 → React 18
Legacy Modern

Four-hour version: emphasize MCP setup, migration plan quality, and one contained implementation slice.

Mini-Lab: Create a Playwright Skill

Capture the workflow once, reuse it every time

Create a skill named playwright-e2e that captures our browser testing conventions:

- Prefer accessible locators
- Use page objects only after duplication appears
- Run `npm playwright test`
- Summarize failures and screenshots

1. Inspect Existing Skills

2. Ask Junie to Create One

3. Use It on a Test

4. Run Local CLI

Teaching point: Playwright MCP is good for interactive browser exploration. A Skill is better for repeatable team testing conventions.

Common Patterns

TDD Flow

1. Ask: "What should fail?"
2. Plan: "How will we prove it?"
3. Code: "Implement and run tests"

Red → Green → Refactor

Plan-First Feature

1. State acceptance criteria
2. Review Junie's plan
3. Approve a focused diff

Scope stays visible

Upgrades

1. Use current docs
2. Plan migration
3. Verify behavior

Systematic migration



CI/CD Integration — Junie GitHub Action

```
name: Junie Agent
on:
  issue_comment:
    types: [created]
  pull_request_review_comment:
    types: [created]

permissions: { contents: write, pull-requests: write, issues: write }

jobs:
  junie:
    if: contains(github.event.comment.body, '@junie-agent')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: JetBrains/junie-github-action@v1
        with:
          junie_api_key: ${ secrets.JUNIE_API_KEY }
```

GitHub: `@junie-agent` in issues or PRs

GitLab: `#junie` in merge request comments

CLI: `junie "prompt"` for headless local automation



Remote Mode Demo

What It Is

- Browser UI for the same CLI session
- Works from laptop, tablet, or phone browser
- Terminal and web UI stay synchronized
- Your machine must stay awake

Demo Flow

1. Start ``junie`` in the terminal
2. Run ``/remote``
3. Open ``junie.jetbrains.com/remote``
4. Respond from the browser
5. Run ``/remote`` and choose Stop Remote Session

Remote mode uses the Junie service. JetBrains Account sign-in is the simplest path; a Junie API key is the alternate route.

🤔 Why Junie Instead of Cursor?

Use Junie When...

- Team lives in JetBrains IDEs
- IDE refactorings matter
- Plans before edits matter
- IDE, CLI, and CI should align

Use Cursor When...

- AI-first editor is desired
- Cursor Tab is central
- Background agents matter
- Team already uses rules/Bugbot

Compare Live

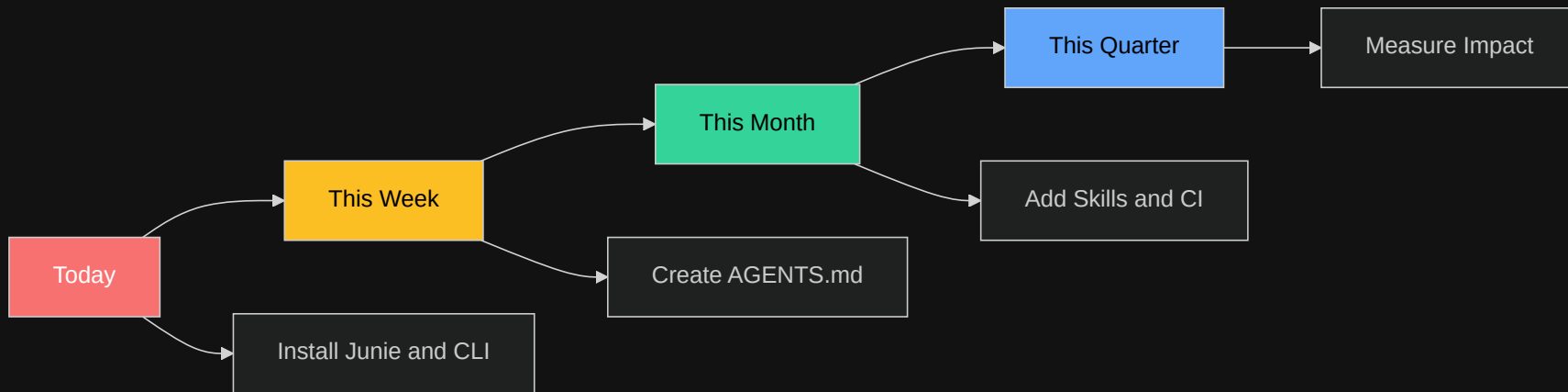
- Same repo
- Same task
- Same acceptance criteria
- Stop before a second workshop

Cursor can also run inside JetBrains through ACP, so this is about workflow fit.

✨ Key Takeaways

- ✓ **Ask before Code** - Understand first, implement second
- ✓ **Plan before big changes** - Align on requirements, tests, and scope
- ✓ **Guidelines drive consistency** - Encode your team's DNA
- ✓ **Skills preserve workflows** - Reuse task-specific patterns
- ✓ **Build trust gradually** - Approvals → Allowlist → Brave
- ✓ **CLI matters** - Use Junie outside the IDE and in automation
- ✓ **Safety first** - Review diffs, run tests, maintain control

Your Next Steps



Start with Ask and Plan → Approve focused diffs → Add guidelines → Automate later

Discussion Time

Let's Explore Together:

-  IDE plugin vs CLI vs CI/CD — which fits your workflow?
-  Where do you always want human review?
-  How could AGENTS.md guidelines help your team?
-  Which MCP tools and Agent Skills would you create?
-  Where does Cursor, Codex, Claude, or Antigravity fit better?

Resources Hub

Documentation

-  [Junie Docs](#)
-  [Guidelines & Memory](#)
-  [Agent Skills](#)
-  [CLI Reference](#)

Agent Workflows

-  [GitHub Action](#)
-  [GitLab CI](#)
-  [Plan Mode](#)
-  [Remote Mode](#)

GitHub

-  [Guidelines Catalog](#)
-  [Context7 MCP](#)
-  [This repository](#)

Adjacent Tools

-  [Cursor Docs](#)
-  [Codex / Claude / Antigravity docs](#)
-  [Agent Communication Protocol](#)

Community

-  [JetBrains AI Slack](#)
-  [Stack Overflow: jetbrains-junie](#)
-  [Support team](#)

 Thank You!

AI amplifies good practices



Remember:

**Good agents amplify clear intent,
tested code, and team conventions.**

Questions? Let's explore together! 

About Ken Kousen

Ken Kousen

President, Kousen IT, Inc.

 ken.kousen@kousenit.com

 github.com/kousen

 [@talesfromthejarside](https://www.talesfromthejarside.com)

 kousenit.substack.com

 linkedin.com/in/kenkousen

 bsky.app/profile/kousenit.com

AI agents aren't here to replace developers—
they're here to make us **better** developers.



Bonus: Quick Reference

IDE Shortcuts

- `Ctrl+Alt+J` - Open Junie
- `Ctrl+Enter` - Execute
- `Esc` - Cancel operation ### CLI

Shortcuts • `Shift+Tab` - Plan mode

- `/mcp` - MCP setup
- `/model` - Choose model
- `/usage` - Credits usage
- `Ctrl+B` - Brave mode
- `Ctrl+R` - Prompt history

Common Commands

"Analyze this file"

"Create a plan first"

"Add tests with 90% coverage"

"Implement and run tests"

"Review the diff"

MCP Tools

```
{  
  "context7": "Library docs",  
  "playwright": "E2E tests",  
  "search": "Web search",  
  "custom": "Your tools"  
}
```



Print this slide for your desk!